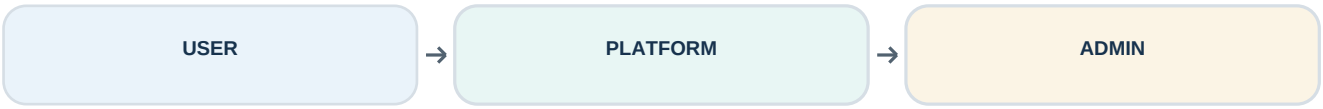
NCC_DSA

CENTRALIZED LEARNING, ATTENDANCE & COMPETITIVE PROGRAMMING PLATFORM

SOFTWARE REQUIREMENTS SPECIFICATION

Detailed system requirements, workflows, data model, security, testing, deployment and acceptance criteria. This edition expands each section so every page is information-dense while preserving comfortable spacing and visual hierarchy.



Document	NCC_DSA Software Requirements Specification
Edition	Detailed Edition — v2.0
Role model	Exactly two roles: USER and ADMIN
Core domains	Lectures • Attendance • Problems • Contests • Rating • Streak • Restriction
Primary stack	Web frontend • Node/Express backend • MongoDB • JWT • Codeforces integration
Deployment	Frontend: Vercel • Backend: Render • Database: MongoDB Atlas

Prepared as the consolidated engineering specification for the NCC_DSA platform.

00 Document Control & Reading Guide

Purpose

This SRS defines what the NCC_DSA platform must do, who can perform each action, what data must be stored, how activities affect progress and rating, and how the system should be validated. It is intended to be usable by developers, testers, administrators and project reviewers.

Source discipline

The requirements below are consolidated from the project material supplied in the conversation. Where implementation details were not explicitly specified, they are presented as recommended engineering structure rather than as a new business rule.

Document map

Part	Coverage
01–05	Vision, scope, actors, architecture, authentication
06–10	Announcements, lectures, attendance, contests, Codeforces
11–15	Progress sheet, verification, rating, streak, restriction
16–20	Admin control, UX, frontend/backend, database
21–24	API, security, validation, non-functional requirements
25–28	Git workflow, development, testing, deployment
29–32	Acceptance, scenarios, risks, traceability

KEY BUSINESS RULE

The Admin controls the process; the User participates in the process; verified activity produces progress; progress, contest performance and attendance contribute to NCC_DSA rating; rating determines participation status according to defined rules.

01 Introduction & Vision

Objective

NCC_DSA is a centralized platform for coordinating learning activities, attendance, competitive programming and measurable student progress in one controlled system. The platform replaces fragmented processes with a consistent lifecycle from participation to verified progress.

Vision

Create a single source of truth for NCC_DSA activities: announcements tell users what is happening; lectures provide learning opportunities; attendance records participation; problems and contests measure practice; verification turns claims into evidence; rating and streaks summarize sustained activity.

Design principles

- Centralized administration
- Verified rather than self-declared progress
- Two-role authorization model
- Traceable activity history
- Clear restriction and reopening workflow
- Simple user-facing participation flow

Success definition

A successful implementation allows an Admin to operate the complete activity lifecycle while a User can participate without administrative privileges. Every important activity has an owner, status, timestamp and auditable outcome.

Section 01 implementation note: keep the business rule server-authoritative, testable and auditable.

02 Product Scope

In scope

Authentication; announcements; lecture management; learning resources; live links; attendance sessions; contest publishing; Codeforces result fetching and matching; progress sheet; problem verification; internal rating; rating history; streak tracking; warnings; restriction below the critical threshold; Admin reopening; audit records.

Out of scope / not defined

The source requirements do not define additional organizational roles beyond USER and ADMIN. They also do not define a separate Mentor, Division Lead, Super Admin or Organizer permission model.

Primary user journey

A User registers, logs in, reads announcements, joins lectures or resources, submits attendance code, solves assigned problems, verifies linked-platform progress, participates in contests and sees the resulting rating/streak status.

Administrative journey

An Admin creates and publishes activities, manages attendance, manages contests and results, controls the progress sheet and verification rules, monitors rating/restrictions, and reopens restricted accounts after review.

Section 02 implementation note: keep the business rule server-authoritative, testable and auditable.

03 Stakeholders & Roles

Roles

Exactly two roles exist: USER and ADMIN.

USER capabilities

Register and authenticate; view announcements; access lectures/resources/live links; submit attendance code; view assigned problems; claim a solved problem; view verification status; view contests/results; view rating, stars, streak and warnings; see restriction instructions.

ADMIN capabilities

Manage announcements, lectures, resources, live links, attendance sessions, contests, invitations, results, progress topics/problems, verification controls, rating values/formula, warnings, restricted users and reopening decisions.

Authorization rule

The backend must enforce role authorization independently of the frontend. Hiding an Admin button is not a security control; every protected API must validate authentication and role.

Section 03 implementation note: keep the business rule server-authoritative, testable and auditable.

04 Functional Architecture

Architecture

The platform follows a layered web architecture. The frontend presents role-aware pages and sends authenticated requests. The backend owns business rules, validation, authorization and integration logic. MongoDB stores the system of record.

Major modules

- Identity & access
- Announcements & lectures
- Attendance
- Contest & Codeforces
- Progress & verification
- Rating & streak
- Restriction & reopening
- Audit and administration

Information flow

User/Admin → Frontend → API → Controller/Service → Database or external platform → normalized response → UI.

Boundary

Business rules such as attendance expiration, duplicate prevention, verification status and rating calculation must be implemented server-side so all clients receive the same result.

Section 04 implementation note: keep the business rule server-authoritative, testable and auditable.

05 Authentication & Authorization

Registration

Users provide the registration data required by the platform. The system creates a USER account after validation and stores credentials securely.

Login

The user submits credentials. On success the backend issues an authenticated token/session mechanism used for protected requests.

JWT/API protection

Protected endpoints require a valid authentication token. Middleware extracts identity, validates the token and attaches the authenticated user context to the request.

Role enforcement

ADMIN-only operations require both valid authentication and role=ADMIN. USER endpoints must not expose administrative mutation operations.

Account states

At minimum, the platform needs an ACTIVE state and a RESTRICTED state. A restricted account cannot continue normal platform participation until an Admin approves reopening.

Section 05 implementation note: keep the business rule server-authoritative, testable and auditable.

06 Announcements

Purpose

Announcements are the primary communication mechanism for upcoming lectures, contests, resources and other NCC_DSA activities.

Admin operations

Create, edit, publish, unpublish/archive and remove announcements according to the application's final moderation policy.

User experience

Users see published announcements in a clear chronological feed with title, message, publication state/time and optional related activity link.

Traceability

Each announcement should retain its creator and timestamps so the system can distinguish current published content from historical content.

Section 06 implementation note: keep the business rule server-authoritative, testable and auditable.

07 Lecture Management

Lecture entity

A lecture should identify its title/topic, description, date/time, instructor or presenter information where applicable, related resources and live link when provided.

Admin workflow

Admin creates the lecture → adds details/resources → optionally creates attendance session → publishes information → users participate.

User workflow

User opens lecture information, accesses the live/resource link and can submit attendance while the attendance session is active.

Relationship

Attendance sessions belong to a lecture. This relationship must be checked by the backend to prevent a valid code from being attached to the wrong lecture.

Section 07 implementation note: keep the business rule server-authoritative, testable and auditable.

08 Attendance

Session creation

When an Admin creates attendance for a lecture, the system generates a four-digit code such as 4827 and creates an attendance session.

Validity

The default validity can be 10 minutes 15 seconds. Admin can change the validity. The server should store a precise start and expiry time.

Submission checks

The backend verifies: authenticated user; attendance session exists; session belongs to the lecture; session is active; code matches; user has not already received an attendance record for that lecture.

Attendance record

Store User, Lecture, Status, Attendance Session, Submitted Time and Verification Result.

Duplicate rule

A user cannot receive multiple attendance records for the same lecture. The database should support this rule with an appropriate uniqueness strategy.

Failure handling

Expired, invalid, duplicate or unauthorized submissions must return clear machine-readable errors and must not create an attendance record.

Section 08 implementation note: keep the business rule server-authoritative, testable and auditable.

09 Contest Management

Creation

Admin creates a contest with title, description, date, time, platform, contest URL and invitation link.

Publication

Admin publishes an announcement so users can discover the contest and join the external platform.

Completion

After the contest, Admin triggers or finalizes result fetching. The system retrieves the configured result data and matches participants using their linked Codeforces handles.

User view

Users can see contest participation/performance such as rank and score where available. Contest performance contributes to the internal NCC_DSA rating according to the configured formula.

Data integrity

A contest result should be associated with a specific contest and participant. Re-fetching should not unintentionally create duplicate result records.

Section 09 implementation note: keep the business rule server-authoritative, testable and auditable.

10 Codeforces Integration

Purpose

The integration connects external Codeforces contest performance with the NCC_DSA platform.

Required mapping

Each participating user must have a stored Codeforces handle or account identifier that can be matched against returned contest results.

Processing flow

Admin selects/finalizes contest → backend calls the external source → receives results → normalizes participant data → matches handles → stores contest results → calculates contest contribution → updates rating history.

Failure cases

External API unavailable, contest not finished, missing handle, unmatched handle, malformed result or duplicate fetch must be handled explicitly.

Auditability

The system should retain enough metadata to identify when results were fetched and which contest they belong to.

Section 10 implementation note: keep the business rule server-authoritative, testable and auditable.

11 Progress Sheet

Purpose

The progress sheet is the structured list of programming tasks that users are expected to practice.

Admin controls

Admin creates and organizes problems by topic, platform, link and task value.

Topic grouping

Examples include Arrays, Binary Search, Dynamic Programming and Graphs. Topics are an organizational mechanism and can be expanded by Admin.

User action

User opens a problem, follows the external problem link, solves it, then clicks “I Solved This”. That action creates a verification claim rather than immediate credit.

Task value

Each problem can have a configurable task value controlled by Admin. Only verified completion should contribute to problem-solving rating.

Section 11 implementation note: keep the business rule server-authoritative, testable and auditable.

12 Problem Verification

Core rule

“I Solved This” is a claim, not proof. The system must verify the claim against the user's linked platform account.

Statuses

NOT SOLVED; VERIFICATION PENDING; SOLVED / VERIFIED.

Verification lifecycle

User claims → backend checks linked account/platform evidence → status becomes VERIFIED when evidence meets the configured rule; otherwise the claim remains pending or returns to an appropriate non-verified state.

Rating rule

Only SOLVED / VERIFIED problems contribute to problem-solving rating.

User visibility

The user should always be able to see the current verification status so they understand whether an activity has actually affected their progress.

Section 12 implementation note: keep the business rule server-authoritative, testable and auditable.

13 Rating System

Definition

NCC_DSA rating is an internal 0–5 normalized measure derived from verified problem solving, contest performance and attendance.

Conceptual formula

Problem Solving + Contest Performance + Attendance = NCC_DSA Rating.

Admin control

Admin controls activity/task values and the conversion/normalization formula.

Example

Verified Problems: 45; Problem Score: 32; Contest Score: 18; Attendance Score: 12; Total: 62; Normalized rating: 3.72/5; Stars: ★★★★■.

Precision and history

The visual star display may be discrete while the numerical rating retains precision such as 4.27/5. Previous rating values must be preserved in history instead of simply overwritten.

Change events

A rating recalculation should create a traceable history event containing previous value, new value, cause/source and timestamp.

Section 13 implementation note: keep the business rule server-authoritative, testable and auditable.

14 Streak Tracking

Definition

A streak represents consecutive days on which a user completes qualifying activities.

Qualifying activity

The streak must be based on actual system-defined qualifying activities, not merely opening the website.

Displayed values

At minimum, show current streak and longest streak, for example current 7 days and longest 14 days.

Calculation

The system should derive streak state from dated qualifying activity records and update it when a new qualifying event is verified.

Transparency

The UI should explain what activities qualify so users can understand why a day was or was not counted.

Section 14 implementation note: keep the business rule server-authoritative, testable and auditable.

15 Warnings & Restrictions

Rating drop warning

When a user's rating decreases, the system sends/displays a warning showing previous rating and current rating.

Critical threshold

If rating becomes less than 2.0, the account becomes RESTRICTED.

Restricted experience

The user loses normal platform access and sees a restriction message containing the current rating and instructions to contact the DSA Heads.

Admin review

Admin sees restricted users, their rating/status and relevant activity history. Admin can approve/open access after review.

Reopening

Approved reopening changes the account back to ACTIVE and records the administrative action. If the Admin does not approve, the account remains restricted.

Section 15 implementation note: keep the business rule server-authoritative, testable and auditable.

16 Admin Control

Central control model

Admin is the central operational role. All platform-wide configuration and activity-management functions described in the requirements are controlled by Admin.

Control domains

Announcements; lectures; resources; live links; attendance; contests; invitations; results; progress sheet; topics/problems; verification; rating; warnings; restrictions; reopening.

Dashboard

A practical Admin dashboard should surface active lectures, attendance sessions, upcoming/past contests, verification queues, restricted users and rating-related events.

Audit trail

Administrative mutations should be logged with actor, action, target, timestamp and result so important decisions can be reviewed later.

Section 16 implementation note: keep the business rule server-authoritative, testable and auditable.

17 User Experience & UI

Principle

The UI should make the next action obvious and keep operational information close to the activity it describes.

User dashboard

Recommended dashboard areas: announcements, upcoming lectures, active attendance, progress summary, contests, rating/stars, streak and account status.

Status language

Use explicit labels such as ACTIVE, EXPIRED, PENDING, VERIFIED, RESTRICTED and APPROVED rather than ambiguous icons alone.

Restriction page

A restricted user should immediately understand current rating, why access is restricted according to system rules, what normal access is unavailable, and how to contact the DSA Heads.

Responsive design

Core flows should remain usable on desktop and mobile-sized screens because attendance-code entry and announcement reading are likely to occur from phones.

Section 17 implementation note: keep the business rule server-authoritative, testable and auditable.

18 Frontend Architecture

Recommended structure

Use feature-oriented organization with shared components and role-aware layouts.

Suggested folders

frontend/src/components; layouts; pages; features; hooks; services; context; utils; types.

Services

Centralize API calls in service modules. Handle token/session state consistently and normalize API errors for the UI.

Route protection

Protected routes should check authentication state and role, but backend authorization remains authoritative.

State

Shared state may include authenticated user, account status, announcements, active attendance session, rating summary and verification state.

Section 18 implementation note: keep the business rule server-authoritative, testable and auditable.

19 Backend Architecture

Recommended structure

backend/src/config; models; controllers; routes; middleware; services; validators; utils; app.js; server.js.

Layer responsibilities

Routes map HTTP endpoints; middleware authenticates/authorizes; controllers coordinate request/response; services hold reusable business logic; models represent persistent data; validators reject malformed input.

Integration services

External Codeforces access should be isolated in a service layer so API-specific behavior does not spread through controllers.

Error middleware

Central error handling should return consistent response shapes and avoid leaking internal stack traces or secrets.

Section 19 implementation note: keep the business rule server-authoritative, testable and auditable.

20 Database Design

Core entities

User, Lecture, Announcement, AttendanceSession, Attendance, Contest, ContestResult, Problem, ProblemProgress, Rating and AuditLog.

Key relationships

Lecture 1→many AttendanceSession; Lecture 1→many Attendance; Contest 1→many ContestResult; Problem 1→many ProblemProgress over users; User 1→many Rating history entries; Admin actions 1→many AuditLog entries.

Integrity rules

Prevent duplicate attendance per user/lecture; identify unique contest/user result pairs; identify unique user/problem progress records where appropriate; retain historical rating events.

Suggested User fields

Identity/contact fields; role; account status; linked Codeforces handle; authentication/security fields; timestamps.

Indexing

Index high-frequency lookup fields such as user identity, role/status, attendance session expiry, lecture references, contest references, platform handles and progress ownership.

Section 20 implementation note: keep the business rule server-authoritative, testable and auditable.

21 API Requirements

API style

Use resource-oriented REST endpoints with consistent JSON response envelopes.

Response contract

A practical response shape is { success, message, data?, error? }, with error codes that allow the frontend to distinguish validation, authentication, authorization, conflict, not-found and integration failures.

Endpoint groups

Auth; announcements; lectures/resources; attendance; contests/results; problems/progress; verification; rating/streak; restrictions; admin dashboard/control.

Authentication

Protected requests carry the configured bearer token/session credential.

Validation

Every mutation endpoint validates required fields, types, ranges and cross-field rules before changing persistent state.

Section 21 implementation note: keep the business rule server-authoritative, testable and auditable.

22 Security Requirements

Credential security

Passwords must never be stored in plaintext. Use a strong password-hashing mechanism and protect authentication secrets through environment configuration.

Authorization

Enforce USER/ADMIN role checks server-side. A USER must not be able to call Admin mutation endpoints simply by constructing an HTTP request.

Token handling

Validate signature, expiry and claims. Do not accept arbitrary user IDs from the client as proof of identity.

Input safety

Validate and sanitize inputs, protect against injection, enforce URL rules where external links are stored, and use secure database access patterns.

Secrets

API keys, JWT secrets and database credentials must be environment variables or secret-manager values, never committed to Git.

Audit and privacy

Record security-sensitive administrative actions while limiting stored personal data to what the platform actually needs.

Section 22 implementation note: keep the business rule server-authoritative, testable and auditable.

23 Validation & Error Handling

Validation layers

Client validation improves UX; server validation is authoritative. Database constraints provide an additional integrity layer.

Business conflicts

Examples: duplicate attendance; expired attendance code; contest result already imported; problem claim already pending; restricted user attempting normal activity.

Response semantics

Return stable HTTP status categories and application error codes. Messages should be understandable to users while logs can contain more technical detail.

Observability

Log request identifiers, error codes and integration failures without logging passwords, raw tokens or other secrets.

Section 23 implementation note: keep the business rule server-authoritative, testable and auditable.

24 Non-Functional Requirements

Performance

Common dashboard and list requests should return quickly under normal load. Expensive external result fetching should be asynchronous or isolated so it does not block unrelated user requests.

Reliability

Critical mutations such as attendance creation, attendance submission and rating history updates should be atomic enough to avoid partial state.

Scalability

Separate frontend, API and database responsibilities so each layer can scale independently.

Maintainability

Use TypeScript or clearly structured JavaScript, modular services, consistent naming, reusable validation and tests around business rules.

Accessibility

Use readable contrast, keyboard-accessible controls, descriptive labels and status text that does not rely solely on color.

Backup/recovery

Database backups and environment recovery procedures should be defined before production use.

Section 24 implementation note: keep the business rule server-authoritative, testable and auditable.

25 Git & Team Workflow

Branching

Use a feature branch for each coherent change. Keep commits focused and descriptive.

Flow

feature branch → local tests → pull request → peer review → development environment → main/production.

Ownership

Avoid multiple people editing the same module without coordination. Agree on API contracts before parallel frontend/backend work.

Pull requests

PRs should explain the change, affected endpoints/components, tests executed and any migration or environment-variable requirements.

Quality gate

Do not merge changes that break authentication, role enforcement, attendance integrity, rating calculation or existing tests.

Section 25 implementation note: keep the business rule server-authoritative, testable and auditable.

26 Development Phases

Phase 1

Foundation: repository structure, environment configuration, database connection, authentication and two-role authorization.

Phase 2

Communication: announcements, lectures, resources and live links.

Phase 3

Attendance: session generation, code validation, expiry, duplicate prevention and history.

Phase 4

Competitive programming: contest management, invitation links, Codeforces mapping and results.

Phase 5

Progress: topic/problem management, solve claims and verification.

Phase 6

Measurement: rating, stars, history, streak, warnings and restriction/reopening.

Phase 7

Hardening: security, tests, deployment, monitoring and acceptance validation.

Section 26 implementation note: keep the business rule server-authoritative, testable and auditable.

27 Testing Strategy

Unit tests

Test pure rating calculations, normalization, streak calculations, validation and status transitions.

API tests

Test authentication, role enforcement, CRUD endpoints, attendance, contests, verification and restriction workflows.

Integration tests

Use a test database/environment to verify database constraints and multi-step business flows.

End-to-end

Simulate: register/login → attend lecture → submit code → solve/verify problem → participate in contest → import result → rating update → rating drop → restriction → Admin reopen.

Regression

Every bug fixed in attendance, authentication, rating or authorization should add a regression test.

Section 27 implementation note: keep the business rule server-authoritative, testable and auditable.

28 Deployment & Environment

Target architecture

React/Vite-style frontend deployed to Vercel; Node/Express backend deployed to Render; MongoDB hosted on MongoDB Atlas.

Environment separation

Development, test and production must use separate configuration and credentials.

Required configuration

Database URI, JWT/auth secrets, external Codeforces integration settings and frontend/backend base URLs should be environment-configured.

Health checks

Backend should expose an appropriate system-health mechanism for operational verification without exposing secrets.

Release process

Run tests → build frontend/backend → deploy → verify health → smoke-test authentication and core workflows → monitor logs.

Section 28 implementation note: keep the business rule server-authoritative, testable and auditable.

29 Acceptance Criteria

Identity

Exactly USER and ADMIN roles are implemented and protected.

Attendance

Admin can create a four-digit attendance session; expiry is enforced; Admin can configure validity; duplicate attendance is prevented.

Contests

Admin can create/publish contests; results can be fetched/finalized; participant handles can be matched; results are visible.

Progress

Admin controls problems/topics/task values; users can claim completion; claims are verified; only verified problems affect rating.

Rating

Rating combines configured problem, contest and attendance contributions; normalized value is shown on a 0–5 scale; history is retained.

Streak

Current and longest streak are shown and derive from qualifying activities.

Restriction

Rating below 2.0 changes account to RESTRICTED; restricted UI explains status; Admin can review and reopen.

Traceability

Important administrative and rating changes are recorded sufficiently for audit/review.

Section 29 implementation note: keep the business rule server-authoritative, testable and auditable.

30 End-to-End Scenarios

Scenario A — Lecture attendance

Admin creates lecture → opens attendance → server generates code and expiry → User logs in → enters code → backend validates session/code/time/duplicate → attendance record created → activity becomes available for rating.

Scenario B — Problem verification

Admin publishes problem → User opens external problem → User solves it → clicks “I Solved This” → verification checks linked account → status becomes VERIFIED → task value contributes to problem-solving score → rating history records change.

Scenario C — Contest result

Admin publishes contest and invitation → users participate externally → Admin triggers result fetch → backend matches Codeforces handles → results stored → contest score calculated → rating recalculated → history updated.

Scenario D — Restriction and reopening

Rating decreases below 2.0 → warning shows previous/current rating → account becomes RESTRICTED → User sees contact instructions → Admin reviews status/history → Admin approves → account returns ACTIVE and reopening is audited.

Section 30 implementation note: keep the business rule server-authoritative, testable and auditable.

31 Risks & Dependencies

External dependency

Codeforces availability, response format, rate limits and participant-handle matching can affect result import and verification.

Data risk

Incorrect duplicate handling can inflate attendance or progress. Database constraints and transaction-safe logic are therefore important.

Rule risk

Rating formulas and qualifying activities may evolve. Keep formula/configuration isolated from UI and preserve historical values.

Operational risk

Admin actions have platform-wide effects. Use confirmation for destructive or high-impact actions and maintain an audit trail.

Deployment risk

Environment variables, database connectivity, CORS and token configuration must be validated in each deployment environment.

Requirements gap

The source material defines the business rules but does not specify every UI field, exact API path, or exact external verification algorithm. Those details should be finalized during implementation without changing the core rules.

Section 31 implementation note: keep the business rule server-authoritative, testable and auditable.

32 Traceability & Final Requirements

Traceability matrix

The following matrix connects the major business rules to implementation areas.

Matrix

Requirement	Primary module	Verification evidence
Two-role authorization	Auth + middleware	Role-protected API tests
Attendance code + expiry	Attendance service	Time/duplicate test cases
Contest result import	Contest + integration service	Mock/external integration tests
Verified problems only	Progress + verification	Status transition tests
0–5 rating + history	Rating service	Calculation/history tests
Streak from qualifying activity	Streak service	Date-boundary tests
< 2.0 restriction	Account status	Restriction/reopen E2E
Admin reopening	Admin control + audit	Approval/action log test

Final principle

The Admin controls the process; the User participates in the process; verified activity produces progress; progress, contest performance and attendance contribute to the NCC_DSA rating; the rating determines the user's participation status according to the defined rules.

Non-negotiable role rule

Exactly two roles only: USER and ADMIN.

Section 32 implementation note: keep the business rule server-authoritative, testable and auditable.